

Resource modeling

COMPLETE LEARNING GUIDE

What You'll Learn

REST is often reduced to 'use nouns for URLs and HTTP verbs for actions,' but resource modeling — deciding what counts as a resource, how resources relate to each other, and how deeply to nest URLs — is where most real API design difficulty actually lives. A well-modeled API reads predictably: once a developer learns the shape of one resource's routes, they can guess the shape of every other resource's routes correctly. This guide covers naming conventions, collection vs. single-resource endpoints, when (and when not) to nest resources under one another, how to represent relationships between resources, and how HTTP methods map onto create/read/update/delete operations with correct idempotency semantics. These decisions are made once, early in a project, and are expensive to change later — getting them right up front matters far more than most other early technical decisions.

1. Resources Are Nouns, Not Actions

The foundational REST convention is that a URL identifies a resource (a thing — a user, an order, a product), and the HTTP method expresses the action performed on it, rather than encoding the action into the URL itself. This means `/api/users/42` (a noun) combined with a DELETE request expresses 'delete user 42,' whereas a URL like `/api/deleteUser?id=42` encodes the verb into the path and ignores the HTTP method entirely — a strong signal of a poorly modeled, RPC-flavored API rather than a RESTful one. Sticking to nouns consistently makes an API's shape predictable across every resource.

- Good: GET `/api/orders/42`, DELETE `/api/orders/42`, PATCH `/api/orders/42`
- Bad: GET `/api/getOrder?id=42`, POST `/api/deleteOrder`, POST `/api/updateOrderStatus`
- Use plural nouns for collections consistently: `/api/users`, not `/api/user`.
- Reserve query parameters for filtering/sorting, not for identifying a specific resource (that's what path parameters are for).

2. Collections vs. Single Resources

Every resource type typically exposes two URL shapes: a collection endpoint (`/api/orders`) representing the set of all orders, and a member endpoint (`/api/orders/{orderId}`) representing one specific order. GET on the collection returns a list (usually paginated); POST on the collection creates a new resource within it. GET on a member returns that one resource; PUT/PATCH updates it; DELETE removes it. This consistent pairing is what lets a client (or a new team member) correctly guess the shape of an endpoint they've never seen before, based on the pattern established by every other resource in the API.

JAVASCRIPT

```
// Collection endpoints
router.get("/api/orders", ordersController.list);      // list orders
router.post("/api/orders", ordersController.create);  // create an order

// Member endpoints
router.get("/api/orders/:orderId", ordersController.getOne);
router.patch("/api/orders/:orderId", ordersController.update);
router.delete("/api/orders/:orderId", ordersController.remove);

// Nested resource: items belonging to one order
router.get("/api/orders/:orderId/items", orderItemsController.list);
```

3. Nesting Resources — and Knowing When to Stop

Nesting a resource under its parent (`/api/orders/:orderId/items`) is appropriate when the child resource cannot meaningfully exist without its parent and is always accessed in that parent's context — an order item doesn't exist independent of an order. It becomes a problem past one or two levels: a URL like `/api/companies/:id/departments/:id/employees/:id/tasks/:id` is hard to construct, hard to read, and usually means the deepest resource (a task) should instead have its own top-level, flatter route (`/api/tasks/:taskId`) with the relationship expressed through a field (`task.employeeId`) rather than the URL path.

- Nest when the child is truly owned by and meaningless without the parent (order != order items).
- Avoid nesting past two levels — flatten deeper relationships into a top-level resource with a foreign-key-style field instead.
- A resource that's accessed both standalone and nested (e.g. `/api/tasks/:id` and `/api/projects/:id/tasks`) should still have one canonical top-level route.
- Nested collection routes still support the same query parameters (filtering, pagination) as top-level ones.

4. Filtering, Sorting, and Pagination Belong in the Query String

Path parameters identify a specific resource; query parameters shape a collection response without creating a new conceptual endpoint. A request for 'pending orders, sorted by date, page 2' should be expressed as `GET /api/orders?status=pending&sort=-placedAt&page=2` — a single flexible collection endpoint — rather than as separate endpoints like `/api/orders/pending` or `/api/pending-orders-sorted-by-date`, which multiply endlessly as filter combinations grow. This keeps the number of routes bounded and makes filtering behavior discoverable and composable.

- `?status=pending` — filter by field value.
- `?sort=-placedAt` — sort descending by `placedAt` (a leading '-' is a common convention for descending).

- `?page=2&limit=20` or `?cursor=abc123` — pagination (offset-based or cursor-based).
- `?fields=id,status,total` — sparse fieldsets, returning only requested fields to reduce payload size.

5. Representing Relationships: Embedding vs. Linking

When one resource references another (an order references a customer), the API has to decide whether to embed the related resource's full data inline, embed just its identifier, or provide a link the client can follow. Embedding full data (`order.customer = { id, name, email }`) saves the client an extra request but risks returning stale or overly large payloads. Returning just an ID (`order.customerId`) keeps responses small and cache-friendly but requires a second request to get customer details. Many production APIs support both, controlled by an `?expand=customer` or `?include=customer` query parameter, letting the client opt into the heavier response only when needed.

6. HTTP Methods, CRUD, and Idempotency

Each HTTP method carries a specific meaning that a well-modeled API should respect exactly, because clients, caches, and proxies all rely on these semantics. GET and DELETE are idempotent — calling them multiple times has the same effect as calling them once — and GET must never have side effects. PUT is defined as a full replacement of a resource and is idempotent (sending the same PUT twice produces the same end state). PATCH is a partial update and is not guaranteed idempotent. POST is used for creation and for actions that don't fit cleanly into CRUD, and is explicitly not idempotent — two identical POSTs are expected to create two resources.

- GET — read, no side effects, idempotent, safe to cache and retry.
- POST — create a new resource (or trigger a non-idempotent action); not idempotent.
- PUT — full replacement of a resource; idempotent (repeating it has no additional effect).
- PATCH — partial update of a resource; not guaranteed idempotent.
- DELETE — remove a resource; idempotent (deleting an already-deleted resource is still 'deleted').

Common Pitfalls

- Encoding verbs into URLs (`/getUser`, `/createOrder`, `POST /deleteOrder`) instead of letting the HTTP method express the action on a noun resource.
- Inconsistent pluralization across resources — `/api/user` alongside `/api/orders` — breaking the pattern clients learn to rely on.
- Nesting resources more than one or two levels deep, producing unreadable, brittle URLs like `/api/a/:id/b/:id/c/:id/d/:id`.
- Creating a new endpoint for every filter combination (`/api/pending-orders`, `/api/orders-this-week`) instead of one collection endpoint with query parameters.

- Using PUT for a partial update (sending only the changed fields) — clients relying on PUT's full-replacement semantics will silently null out omitted fields.
- Exposing raw sequential database IDs without considering whether opaque or UUID identifiers better fit the resource's security/enumeration concerns.

Key Takeaways

- URLs should identify resources (nouns); HTTP methods should express the action — never encode a verb into the path.
- Every resource typically needs both a collection route and a member route, following the same pattern across all resources.
- Nest resources only when the child is truly owned by and meaningless without the parent, and stop nesting past one or two levels.
- Filtering, sorting, and pagination belong in the query string on one flexible collection endpoint, not in a proliferation of specialized endpoints.
- GET, PUT, and DELETE must be idempotent; POST and PATCH are not — clients and caches depend on these guarantees being honored.

Learn More

RESTful API Design — restfulapi.net

<https://restfulapi.net/resource-naming/>

MDN — HTTP Request Methods

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>

Microsoft Learn — Web API Design Best Practices

<https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>